Software Requirements and Design Document

For

Closet Design Game

Version 2.0

Authors:
Vanessa Campoe
Camryn Morris
Chloe Patrick
Wesley Jordan
Jessika Tyler
Ethan McConnell

1. Overview (5 points)

Closet Design Game is a browser-based clothing customization application that lets users explore historical clothing and design looks on a mannequin. The studio has controls for filtering garments by era, region, and category; choosing a mannequin frame and body type; and drawing on a central canvas. The project aims to make fashion exploration interactive by combining selectable clothing templates with digital drawing tools.

The current application is built with React and Fabric.js. It includes nine mannequin combinations (feminine, masculine, or neutral frames, each with slim, average, or broad body types), three catalog garments, brush controls, an eraser mode, and a browser-local collection of saved outfits. The project README also proposes cut-and-paste customization and social-media sharing, which remain future functionality. The system currently runs entirely in the browser, without an account system or remote database.

2. Functional Requirements (10 points)

FR-01 (High). The system shall display a design studio with a navigation bar, a sidebar of garment/mannequin controls, and a central drawing canvas.

FR-02 (High). The system shall allow the user to select exactly one mannequin frame from feminine, masculine, and neutral, and one body type from slim, average, and broad.

FR-03 (High). The system shall display the mannequin SVG matching the selected frame and body type, replacing the previously displayed mannequin.

FR-04 (High). The system shall present era options obtained from the garment catalog, plus an All option that removes the era restriction.

FR-05 (High). The system shall present region options obtained from the garment catalog, plus an All option that removes the region restriction.

FR-06 (High). The system shall allow filtering by All, Tops, Bottoms, and Hats (the headwear catalog category).

FR-07 (High). The system shall apply the selected era, region, and category restrictions together and list only matching garments; when none match, it shall display a no-results message.

FR-08 (High). For every listed garment, the system shall display its name, era, and region and allow the user to select it.

FR-09 (High). When a garment is selected, the system shall show a category-appropriate garment preview on the mannequin canvas. The current implementation uses rectangular placeholder previews.

FR-10 (High). The system shall let the user create freehand strokes on the canvas using a brush.

FR-11 (High). The system shall let the user set brush width from 1 to 50, select a color, and set opacity from 0.10 to 1.00 in increments of 0.05.

FR-12 (High). The system shall let the user toggle between brush mode and eraser mode. The current eraser behavior paints using the canvas background color.

FR-13 (High). The system shall restrict newly drawn strokes to the garment painting boundary. The current boundary is fixed; updating it for each selected garment is a pending refinement.

FR-14 (Medium). The system shall maintain an ordered display of mannequin, bottoms, tops, headwear, and painting boundary so the components appear in the intended front-to-back order.

FR-15 (High). The system shall save a named outfit in the browser, including the mannequin frame, body type, selected garment identifier, serialized canvas content, and creation timestamp. If no name is entered, it shall generate a numbered name.

FR-16 (High). The system shall show saved outfits in a modal list with each outfit name, mannequin settings, and creation date; it shall allow the user to load one into the studio.

FR-17 (High). The system shall allow the user to delete an individual saved outfit from the browser-local collection.

FR-18 (Medium). The completed system shall load and position the actual garment SVG identified by its catalog path and anchor coordinates in place of category-based placeholder shapes. This is planned.

FR-19 (Low). The completed system shall provide cut-and-paste-style editing of selected clothing templates. This is proposed in the README and not yet implemented.

FR-20 (Low). The completed system shall allow the user to share an outfit image to an external social platform. This is proposed in the README and not yet implemented.

3. Non-functional Requirements (10 points)

NFR-01 — Platform. The design studio shall operate as a client-side web application in a current desktop browser with JavaScript, HTML canvas, SVG, and browser storage enabled.

NFR-02 — Display. The design surface shall have a consistent logical size of 600 by 800 pixels, independently of the number of drawn objects.

NFR-03 — Performance. The target interactive drawing performance shall be at least 30 frames per second during ordinary use on a supported laptop or desktop; this target requires performance testing.

NFR-04 — Persistence. An outfit explicitly saved by the user shall remain available after page refresh or browser restart as long as the same browser profile retains localStorage data.

NFR-05 — Privacy. Outfit data shall be stored locally in the user's browser by default; the current version shall not require account registration or upload saved designs to a server.

NFR-06 — Reliability. If saved-outfit storage contains malformed JSON or is unavailable for reading, the app shall avoid a page-level crash and present an empty list or an understandable error.

NFR-07 — Error handling. If a save cannot complete (for example, because browser storage is full or disabled), the completed system shall notify the user rather than silently claiming success. This is a future improvement.

NFR-08 — Data integrity. Serialized drawing data shall be JSON-based, contain an objects array, preserve drawing object order, and include a version marker and selected garment identifiers.

NFR-09 — Usability. The controls for frame, body type, filtering, drawing settings, and saved looks shall have visible labels and indicate their current selection.

NFR-10 — Accessibility. The completed interface should support keyboard access to standard controls, clear focus indication, and readable contrast; full accessibility conformance has not been established.

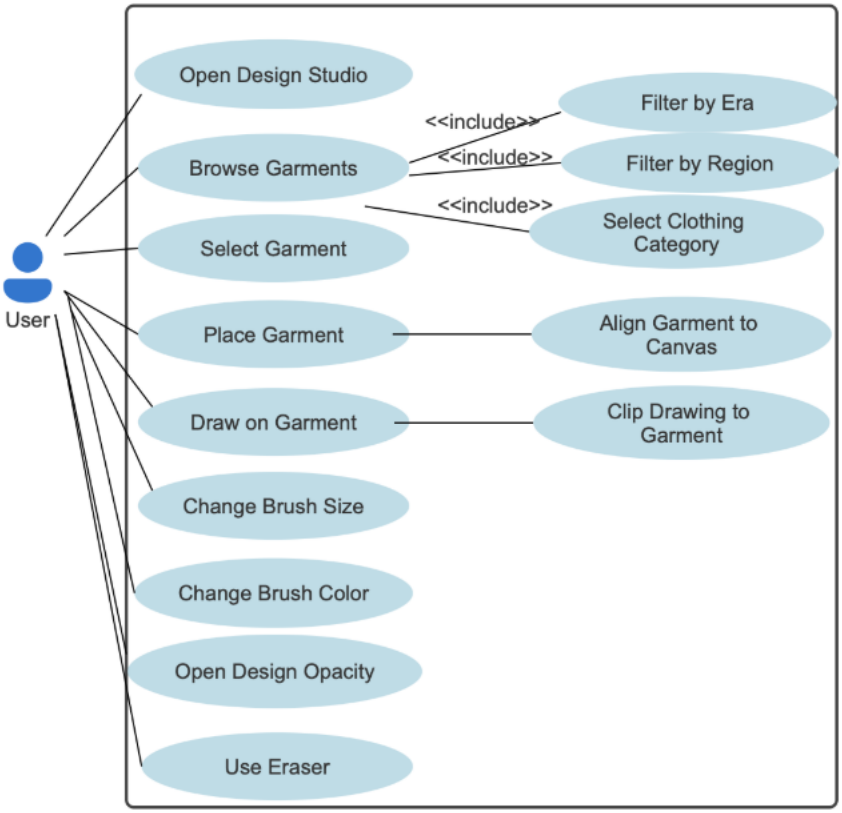
NFR-11 — Maintainability. The source code shall separate the studio component, drawing canvas, saved-outfit dialog, catalog data, and storage/serialization functions into independently maintainable modules.

NFR-12 — Testability. Automated tests shall cover core outfit storage, canvas serialization, restoration, and malformed-data handling; the project shall provide repeatable test and production-build commands.

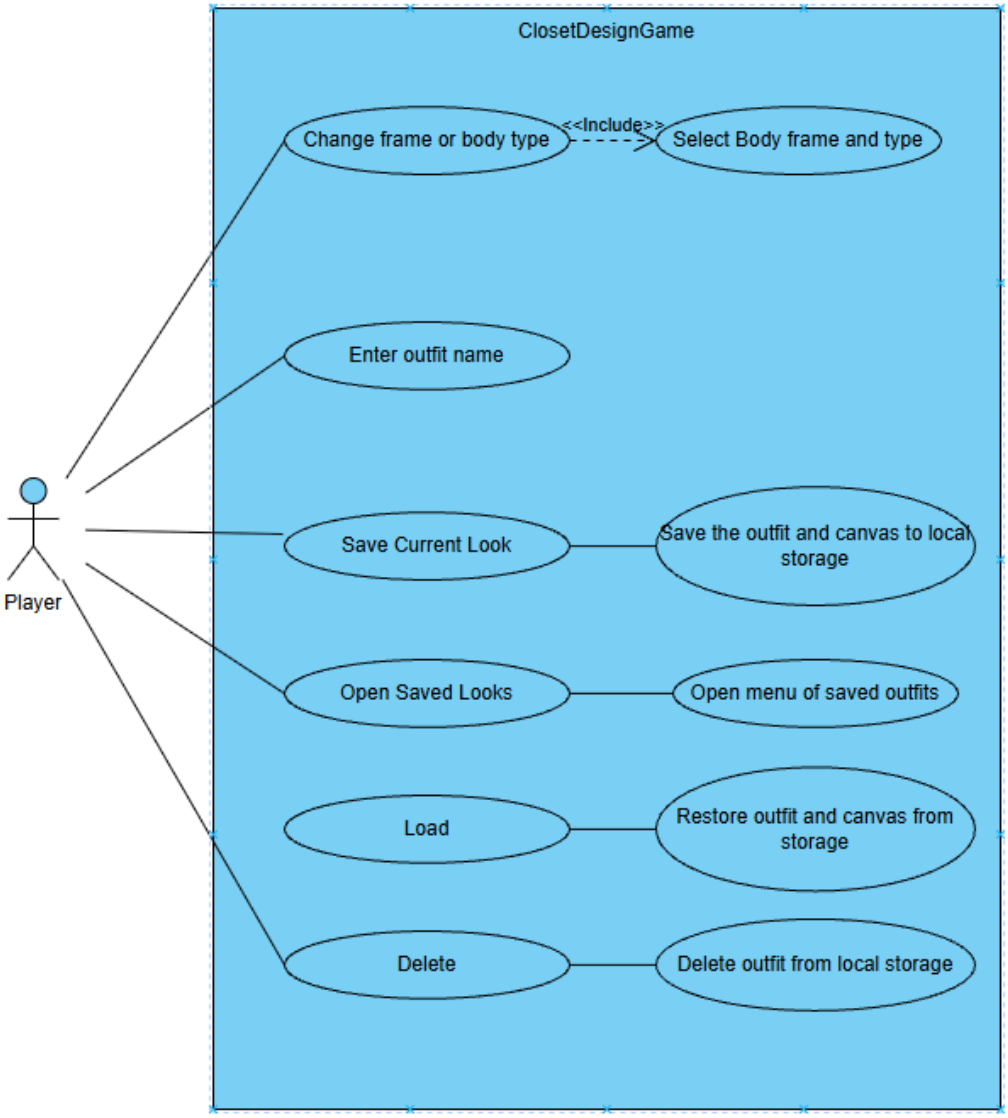
4. Use Case Diagram (10 points)

The following use case diagram presents the primary interactions between a user and the Closet Design Game during Increment.

In the first increment, it was made so the user can access the design studio, browse and filter available garments, select and place garments on the design canvas, and customize garments using the available drawing tools. Drawing controls include brush size, color, opacity, and an eraser. The system also aligns garment assets to the shared canvas coordinate system and clips freehand drawing to the selected garment boundary.

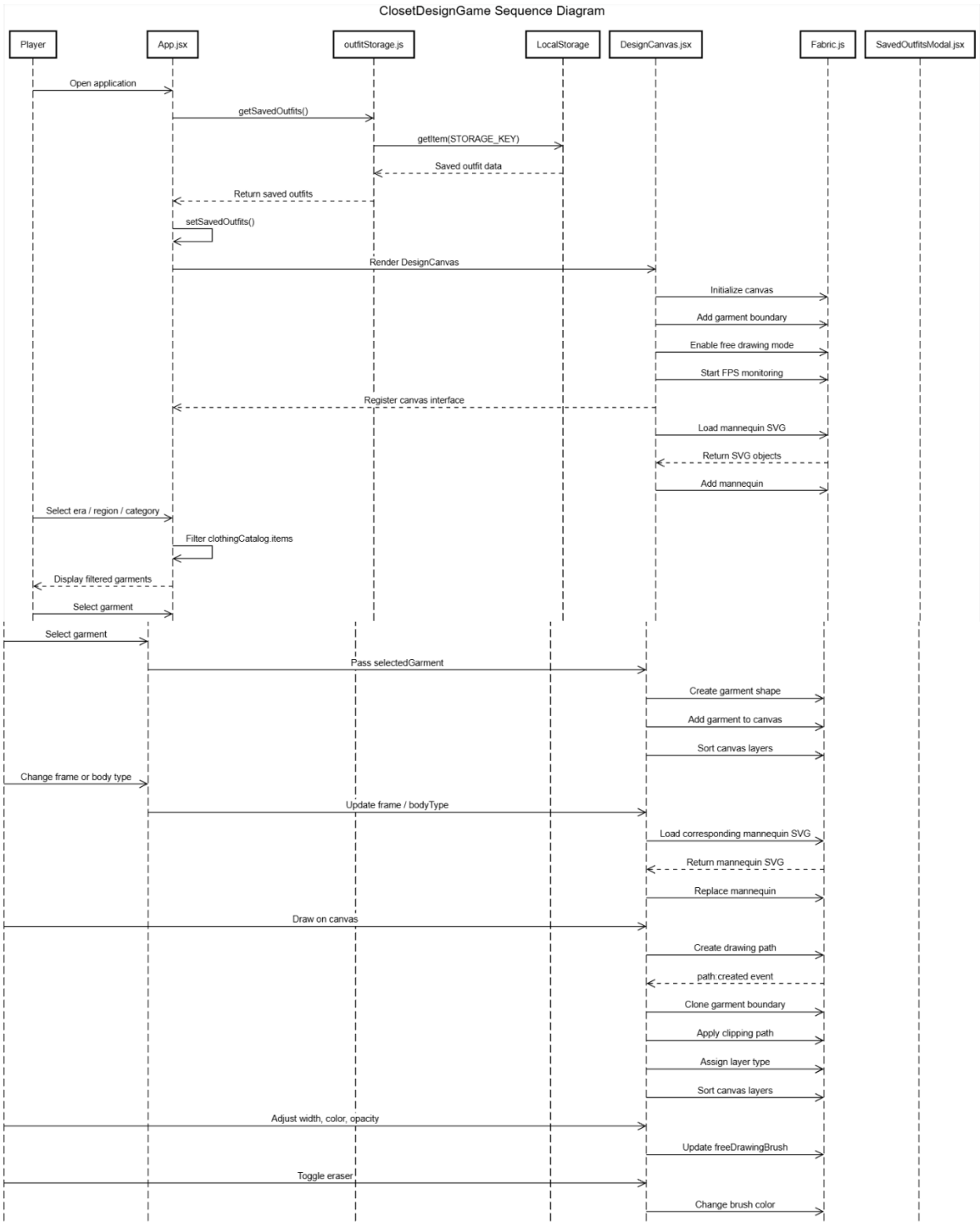


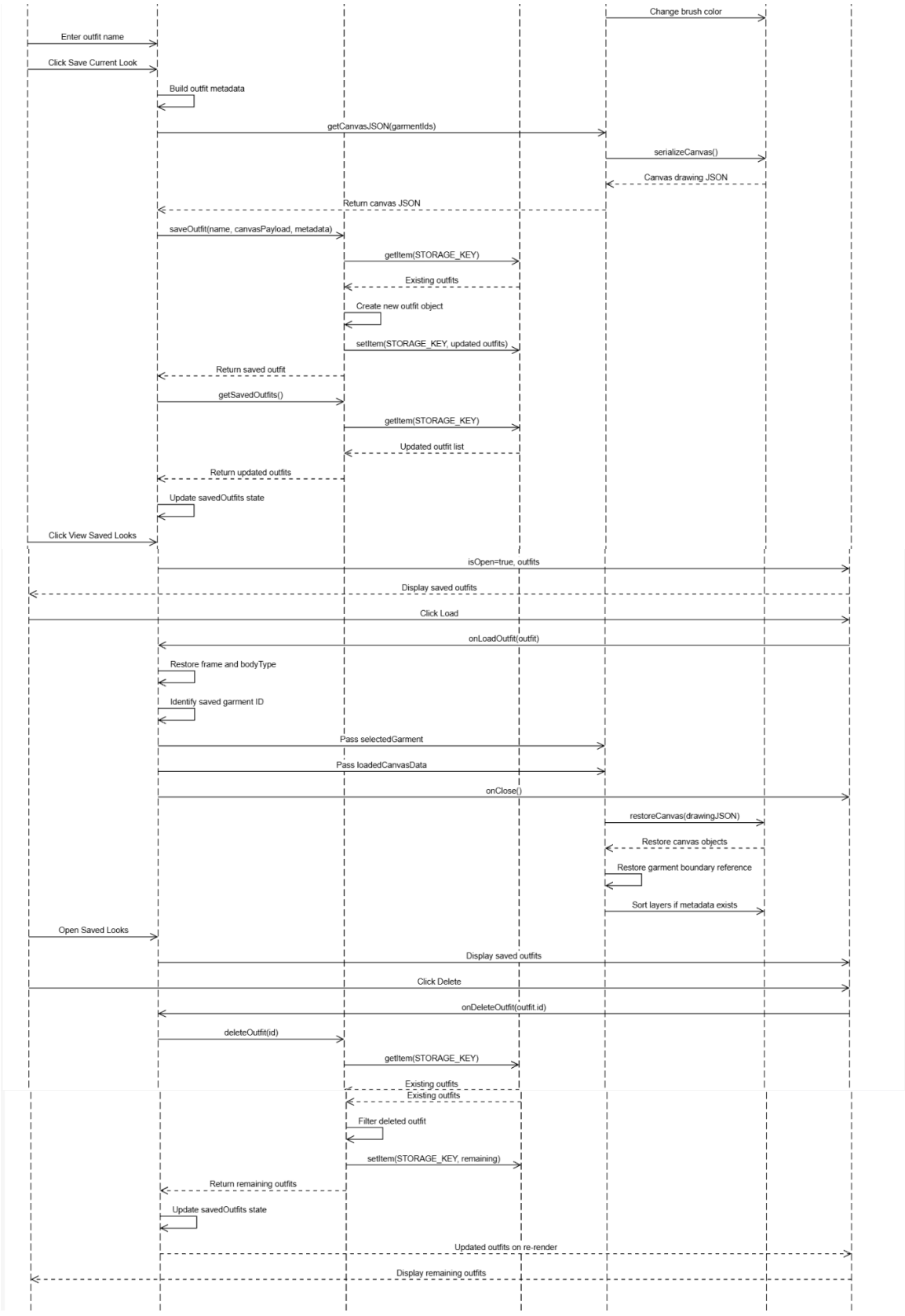
The second increment made it so the user may change the frame or body type of the mannequin, Enter the outfits name, save the current look, open the saved looks menu, load an outfit from the saved looks menu, and delete an outfit from the saved looks menu.



5. Class Diagram and/or Sequence Diagrams (15 points)

The sequential diagram:


ClosetDesignGame Sequence Diagram
Player
App.jsx
outfitStorage.js
LocalStorage
DesignCanvas.jsx
Fabric.js
SavedOutfitsModal.jsx
Open application
getSavedOutfits()
getItem(STORAGE_KEY)
Saved outfit data
Return saved outfits
setSavedOutfits()
Render DesignCanvas
Initialize canvas
Add garment boundary
Enable free drawing mode
Start FPS monitoring
Register canvas interface
Load mannequin SVG
Return SVG objects
Add mannequin
Select era / region / category
Filter clothingCatalog.items
Display filtered garments
Select garment
Select garment
Pass selectedGarment
Create garment shape
Add garment to canvas
Sort canvas layers
Change frame or body type
Update frame / bodyType
Load corresponding mannequin SVG
Return mannequin SVG
Replace mannequin
Draw on canvas
Create drawing path
path:created event
Clone garment boundary
Apply clipping path
Assign layer type
Sort canvas layers
Adjust width, color, opacity
Update freeDrawingBrush
Toggle eraser
Change brush color


Change brush color
Enter outfit name
Click Save Current Look
Build outfit metadata
getCanvasJSON(garmentIds)
serializeCanvas()
Canvas drawing JSON
Return canvas JSON
saveOutfit(name, canvasPayload, metadata)
getItem(STORAGE_KEY)
Existing outfits
Create new outfit object
setItem(STORAGE_KEY, updated outfits)
Return saved outfit
getSavedOutfits()
getItem(STORAGE_KEY)
Updated outfit list
Return updated outfits
Update savedOutfits state
Click View Saved Looks
isOpen=true, outfits
Display saved outfits
Click Load
onLoadOutfit(outfit)
Restore frame and bodyType
Identify saved garment ID
Pass selectedGarment
Pass loadedCanvasData
onClose()
restoreCanvas(drawingJSON)
Restore canvas objects
Restore garment boundary reference
Sort layers if metadata exists
Open Saved Looks
Display saved outfits
Click Delete
onDeleteOutfit(outfit.id)
deleteOutfit(id)
getItem(STORAGE_KEY)
Existing outfits
Existing outfits
Filter deleted outfit
setItem(STORAGE_KEY, remaining)
Return remaining outfits
Update savedOutfits state
Updated outfits on re-render
Display remaining outfits

6. Operating Environment (5 points)

Deployment platform: a desktop or laptop computer running a current graphical web browser. The proposed target operating systems are Windows 11, macOS 14 or later, and Ubuntu 22.04 LTS or later. Chrome, Edge, Firefox, and Safari are suitable browser targets; the repository does not contain evidence of a completed browser/OS compatibility test matrix. A mouse, trackpad, or other pointer input is expected for the current freehand-drawing feature.

Runtime: the application is a web front end using React 19, React DOM 19, Fabric.js 7, Tailwind CSS 4, and Lucide React icons. Mannequin and garment assets are SVG files served with the application. Saved outfits use browser localStorage; no backend service, database, authentication service, or external-sharing API is currently configured.

Development and testing: JavaScript ES modules, Vite 8, npm/Node.js compatible with Vite 8, Vitest 5 with jsdom, and Oxlint. The package scripts include dev, build, preview, lint, test, test:watch, and test:coverage. The 600-by-800-pixel canvas and permanent left sidebar are primarily suited to desktop-width screens.

7. Assumptions and Dependencies (5 points)

Assumptions:

AS-01. The browser supports modern JavaScript, the Canvas API, SVG rendering, and localStorage.

AS-02. The mannequin SVG file for each of the nine frame/body-type combinations remains available under the specified assets directory.

AS-03. Garment catalog records contain valid identifiers, names, eras, regions, categories, SVG paths, and placement metadata.

AS-04. Each user works locally in one browser profile; synchronization across browsers, devices, or users is outside the current scope.

AS-05. Users understand that clearing site data, switching devices, or using private browsing may remove or hide saved designs.

AS-06. Browser storage provides enough space for the expected number and complexity of saved canvas drawings.

AS-07. The current project can remain client-side; account services and remote storage are not necessary to deliver core drawing and saved-look features.

AS-08. If social sharing is added later, the chosen platforms will allow the proposed sharing workflow and the project will comply with their API or share-link requirements.

AS-09. External packages (React, Fabric.js, Tailwind, Vite, Lucide, and Vitest) remain available and compatible with the project's dependency constraints.

Dependencies:

Runtime: React, React DOM, Fabric.js, Tailwind CSS, and Lucide React.

Build/test: Vite, npm/Node.js, Vitest, jsdom, and Oxlint.

Data and assets: src/data/clothingCatalog.json, SVGs in public/assets, and browser localStorage. Social-media APIs/share links would be new dependencies if sharing is implemented.